برنامه نویسی به زبان سی

آرایه ها

دکتر مهران صفایانی

safayani@iut.ac.ir

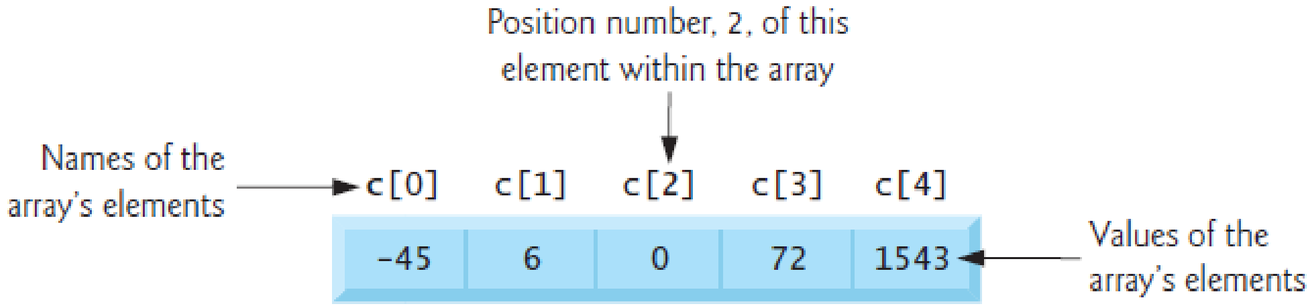
safayani.iut.ac.ir

https://www.aparat.com/mehran.safayani

https://github.com/safayani/Programming_Basics_course

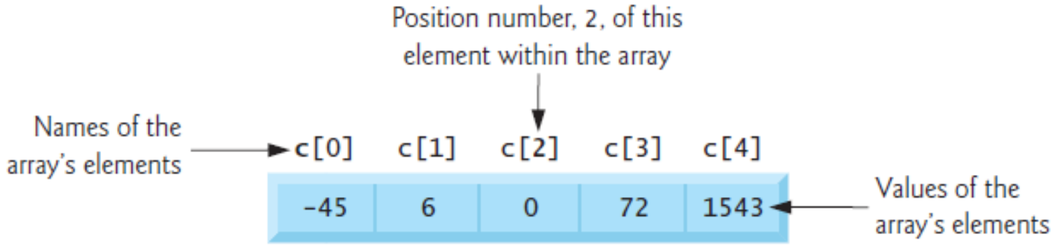
دانشکده مهندسی برق و کامپیوتر، دانشگاه صنعتی اصفهان، اصفهان، ایران

Arrays



- **What is an Array?**
  - A collection of elements of the **same data type**.
  - Stored in a **contiguous** (unbroken) block of memory.
- **Accessing Elements**
  - Use the array name and a **subscript** (or **index**) in square brackets [].
  - **Key Rule:** Array indexing **always starts at 0**.
- **Visual Example (Integer array c)**
  - To access the first element: c[0] (Value is -45)
  - To access the third element: c[2] (Value is 0)

Working with Array Elements



- **Modifying Elements (Writing)**
  - An indexed array element can be assigned a new value.
  - **Example:** c[2] = 1000; *(Changes the value at index 2 to 1000)*
- **Using Elements in Expressions (Reading)**
  - Use elements in calculations just like any other variable.
  - **Example 1:** printf("%d", c[0] + c[1]);
  - **Example 2:** x = c[3] / 2;
- **Key Technical Rules**
  - A subscript must be a non-negative integer (or an expression that results in one).
  - The array subscript operator [] has the **highest level of precedence** and is evaluated first in most expressions.

Operators	Grouping	Type
[] () ++ *(postfix)* -- *(postfix)*	left to right	highest
+ - ! ++ *(prefix)* -- *(prefix)* *(type)*	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
&&	left to right	logical AND
||	left to right	logical OR
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment
,	left to right	comma

Defining an Array

- **Purpose of Definition**
  - To tell the compiler the array's **data type** and **number of elements**.
  - This allows the compiler to reserve the correct amount of memory *before* the program runs.
- **Syntax**
  - DataType ArrayName[NumberOfElements];
- **Examples:** int c[5]; int b[100];
- **The Indexing Rule**
  - An array defined with N elements has valid indices from **0 to N-1**.
  - int c[5]; -> Has valid indices c[0], c[1], c[2], c[3], and c[4].
  - int b[100]; -> Has valid indices from b[0] to b[99].

Array Examples: Defining an Array and Using a Loop to Set the Array's Element Values

```
1  // fig06_01.c
2  // Initializing the elements of an array to zeros.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main(void) {
7     int n[5]; // n is an array of five integers
8
9     // set elements of array n to 0
10    for (size_t i = 0; i < 5; ++i) {
11       n[i] = 0; // set element at location i to 0
12    }
13
14    printf("%s%8s\n", "Element", "Value");
15
16    // output contents of array n in tabular format
17    for (size_t i = 0; i < 5; ++i) {
18       printf("%7zu%8d\n", i, n[i]);
19    }
20 }
```

```
Element   Value
      0       0
      1       0
      2       0
      3       0
      4       0
```

Fig. 6.1 | Initializing the elements of an array to zeros.

size_t: unsigned integral type defined by the C standard

Initializing an Array in a Definition with an Initializer List

```c
// fig06_02.c
// Initializing the elements of an array with an initializer list.
#include <stdio.h>

// function main begins program execution
int main(void) {
   int n[5] = {32, 27, 64, 18, 95}; // initialize n with initializer list

   printf("%s%8s\n", "Element", "Value");

   // output contents of array in tabular format
   for (size_t i = 0; i < 5; ++i) {
      printf("%7zu%8d\n", i, n[i]);
   }
}
```

```
Element   Value
      0      32
      1      27
      2      64
      3      18
      4      95
```

Fig. 6.2 | Initializing the elements of an array with an initializer list.

Common Initialization Methods

Method	Example	Result
Provide exact size and values	`int n[3] = {32, 27, 64};`	`n[0]=32, n[1]=27, n[2]=64`
Let compiler determine size	`int n[] = {1, 2, 3, 4, 5};`	Creates a 5-element array.
Initialize all to zero	`int n[5] = {0};`	All 5 elements are set to `0`.

Using Symbolic Constants for Array Sizes

```c
// fig06_03.c
// Initializing the elements of array s to the even integers from 2 to 10.
#include <stdio.h>
#define SIZE 5 // maximum size of array

// function main begins program execution
int main(void) {
   // symbolic constant SIZE can be used to specify array size
   int s[SIZE] = {0}; // array s has SIZE elements

   for (size_t j = 0; j < SIZE; ++j) { // set the values
      s[j] = 2 + 2 * j;
   }

   printf("%s%8s\n", "Element", "Value");

   // output contents of array s in tabular format
   for (size_t j = 0; j < SIZE; ++j) {
      printf("%7zu%8d\n", j, s[j]);
   }
}
```

```
Element   Value
      0       2
      1       4
      2       6
      3       8
      4      10
```

Fig. 6.3 | Initializing the elements of array `s` to the even integers from 2 to 10.

Using Symbolic Constants for Array Sizes

- **Purpose:** Enhance program readability and maintainability by replacing "magic numbers" with meaningful names. Changing one #define value updates all occurrences automatically.
- **Key Rules:**
  - **No Semicolon:** #define SIZE 5 ✓ (not 5; ✗)
  - **Not a Variable:** Cannot assign values (SIZE = 10 ✗)
  - **UPPERCASE Naming:** Convention for visibility (SIZE, MAX_ELEMENTS)
- **How It Works:** Preprocessor replaces all SIZE occurrences with 5 before compilation. Modify once in #define to change array size throughout entire program.
- **Benefit:** Single modification point improves maintainability and reduces errors when scaling array sizes.

Computing the sum of the elements of an array

```c
// fig06_04.c
// Computing the sum of the elements of an array.
#include <stdio.h>
#define SIZE 5

// function main begins program execution
int main(void) {
   // use an initializer list to initialize the array
   int a[SIZE] = {1, 2, 3, 4, 5};
   int total = 0; // sum of array

   // sum contents of array a
   for (size_t i = 0; i < SIZE; ++i) {
      total += a[i];
   }

   printf("The total of a's values is %d\n", total);
}
```

```
The total of a's values is 15
```

Fig. 6.4 | Computing the sum of the elements of an array.

Objective: Process 20 student ratings (1-5 scale) by counting occurrences of each rating using parallel arrays.

```c
// fig06_05.c
// Analyzing a student poll.
#include <stdio.h>
#define RESPONSES_SIZE 20 // define array sizes
#define FREQUENCY_SIZE 6

// function main begins program execution
int main(void) {
   // place the survey responses in the responses array
   int responses[RESPONSES_SIZE] =
      {1, 2, 5, 4, 3, 5, 2, 1, 3, 1, 4, 3, 3, 3, 2, 3, 3, 2, 2, 5};

   // initialize frequency counters to 0
   int frequency[FREQUENCY_SIZE] = {0};

   // for each answer, select the value of an element of the array
   // responses and use that value as a subscript into the array
   // frequency to determine the element to increment
   for (size_t answer = 0; answer < RESPONSES_SIZE; ++answer) {
      ++frequency[responses[answer]];
   }

   // display results
   printf("%s%12s\n", "Rating", "Frequency");

   // output the frequencies in a tabular format
   for (size_t rating = 1; rating < FREQUENCY_SIZE; ++rating) {
      printf("%6zu%12d\n", rating, frequency[rating]);
   }
}
```

```
Rating    Frequency
     1            3
     2            5
     3            7
     4            2
     5            3
```

Fig. 6.5 | Analyzing a student poll.

Character Arrays for String Storage

- **String Initialization**
- Automatic size calculation with string literal:

```
char string1[] = "first"; // 6 elements: 'f','i','r','s','t',’\0’
```

- Explicit character-by-character initialization:

```
char string1[] = {'f','i','r','s','t','\0’};
```

- **Critical:** Always reserve space for terminating null character \0

String Input/Output Operations

- **Safe Input with scanf**

```
char string2[20];          // Capacity: 19 chars + \0
scanf("%19s", string2);    // Field width prevents overflow
```

- No & needed before array name
- Stops reading at whitespace
- **Output with printf**

```
printf("%s", string2);     // Prints until \0 encountered
```

- **Warning:** Missing \0 causes undefined behavior

Security Risks & Best Practices

- **Buffer Overflow Danger**
- scanf("%s", string2) without field width → severe security vulnerability
- Excess characters overwrite adjacent memory
- Can crash program or create security exploits
- **Defensive Programming**
- Always specify field width in scanf conversion specifier
- Ensure array size > max input length + 1 (for \0)
- Validate input length before processing

Character Access & String Processing

- Use array subscript notation:

```
char first_char = string1[0];  // 'f'
char fourth_char = string1[3]; // 's'
char null_char = string1[5];   // '\0'
```

- **Safe String Traversal**

```
for (int i = 0; i < array_size && string1[i] != '\0'; i++) {
    printf("%c ", string1[i]);
}
```

- Checks both array bounds and null terminator
- Prevents reading beyond valid data

```
Enter a string (no longer than 19 characters): Hello there
string1 is: Hello
string2 is: string literal
string1 with spaces between characters is:
H e l l o
```

Fig. 6.8 | Treating character arrays as strings. (Part 2 of 2.)

```c
// fig06_08.c
// Treating character arrays as strings.
#include <stdio.h>
#define SIZE 20

// function main begins program execution
int main(void) {
   char string1[SIZE] = ""; // reserves 20 characters
   char string2[] = "string literal"; // reserves 15 characters

   // prompt for string from user then read it into array string1
   printf("%s", "Enter a string (no longer than 19 characters): ");
   scanf("%19s", string1); // input no more than 19 characters

   // output strings
   printf("string1 is: %s\nstring2 is: %s\n", string1, string2);
   puts("string1 with spaces between characters is:");

   // output characters until null character is reached
   for (size_t i = 0; i < SIZE && string1[i] != '\0'; ++i) {
      printf("%c ", string1[i]);
   }

   puts("");
}
```

Static and automatic local arrays

```
20 // function to demonstrate a static local array
21 void staticArrayInit(void) {
22    // initializes elements to 0 before the function is called
23    static int array1[3];
24
25    puts("\nValues on entering staticArrayInit:");
26
27    // output contents of array1
28    for (size_t i = 0; i <= 2; ++i) {
29       printf("array1[%zu] = %d  ", i, array1[i]);
30    }
31
32     puts("\nValues on exiting staticArrayInit:");
33
34     // modify and output contents of array1
35     for (size_t i = 0; i <= 2; ++i) {
36        printf("array1[%zu] = %d  ", i, array1[i] += 5);
37     }
38  }
39
```

```
40  // function to demonstrate an automatic local array
41  void automaticArrayInit(void) {
42     // initializes elements each time function is called
43     int array2[3] = {1, 2, 3};
44
45     puts("\n\nValues on entering automaticArrayInit:");
46
47     // output contents of array2
48     for (size_t i = 0; i <= 2; ++i) {
49        printf("array2[%zu] = %d  ", i, array2[i]);
50     }
51
52     puts("\nValues on exiting automaticArrayInit:");
53
54     // modify and output contents of array2
55     for (size_t i = 0; i <= 2; ++i) {
56        printf("array2[%zu] = %d  ", i, array2[i] += 5);
57     }
58  }
```

```c
// fig06_09.c
// Static arrays are initialized to zero if not explicitly initialized.
#include <stdio.h>

void staticArrayInit(void); // function prototype
void automaticArrayInit(void); // function prototype

// function main begins program execution
int main(void) {
   puts("First call to each function:");
   staticArrayInit();
   automaticArrayInit();

   puts("\n\nSecond call to each function:");
   staticArrayInit();
   automaticArrayInit();
   puts("");
}
```

```
First call to each function:

Values on entering staticArrayInit:
array1[0] = 0  array1[1] = 0  array1[2] = 0
Values on exiting staticArrayInit:
array1[0] = 5  array1[1] = 5  array1[2] = 5

Values on entering automaticArrayInit:
array2[0] = 1  array2[1] = 2  array2[2] = 3
Values on exiting automaticArrayInit:
array2[0] = 6  array2[1] = 7  array2[2] = 8

Second call to each function:

Values on entering staticArrayInit:
array1[0] = 5  array1[1] = 5  array1[2] = 5 — values preserved from last call
Values on exiting staticArrayInit:
array1[0] = 10  array1[1] = 10  array1[2] = 10

Values on entering automaticArrayInit:
array2[0] = 1  array2[1] = 2  array2[2] = 3 — values reinitialized after last call
Values on exiting automaticArrayInit:
array2[0] = 6  array2[1] = 7  array2[2] = 8
```

Fig. 6.9 | Static arrays are initialized to zero if not explicitly initialized. (Part 2 of 2.)

static vs. Automatic Local Arrays

- The static keyword changes a local array's **lifetime** from temporary (existing only during a function call) to permanent (existing for the program's entire duration), allowing it to **retain its values** between function calls.

Feature	Automatic Array (Default)	static Array
Lifetime	**Temporary:** Created when the function is called and destroyed when it exits.	**Permanent:** Created once at program startup and exists until the program ends.
Initialization	**Every Time** the function is called.	**Only Once** when the program starts.
Default Value	Garbage / Undefined values if not explicitly initialized.	**Zero** by default.
State Between Calls	**Lost / Reset** after the function returns.	**Retained / Persistent** across multiple function calls.

Passing Arrays to Functions in C

- **1. Passing an Entire Array (Pass-by-Reference)**
- **How it Works:** The array's name is the memory address of its first element. When you pass the name, you are giving the function direct access to the original array's location.
- **Function Call (Syntax):**
  - Pass the array name **without** brackets.
  - modifyArray(hourlyTemperatures, size);
- **Function Definition (Syntax):**
  - Declare the parameter with empty brackets [].
  - void modifyArray(int b[], size_t size)
- **Result:**
  - **Changes made inside the function are PERMANENT and modify the original array in the caller.**
  - **Benefit:** This is highly efficient as it avoids making a time-consuming copy of a large array.

Passing Arrays to Functions in C

- **2. Passing a Single Element (Pass-by-Value)**
- **How it Works:** This behaves like any normal variable. The function receives a **copy** of the element's value, not a reference to the element itself.
- **Function Call (Syntax):**
  - Pass the specific element using its index.
  - modifyElement(hourlyTemperatures[3]);
- **Function Definition (Syntax):**
  - Declare the parameter as a single scalar type.
  - void modifyElement(int element)
- **Result:**
  - **Changes made inside the function only affect the local copy and DO NOT modify the original array element**

```c
// fig06_11.c
// Passing arrays and individual array elements to functions.
#include <stdio.h>
#define SIZE 5

// function prototypes
void modifyArray(int b[], size_t size);
void modifyElement(int e);

// function main begins program execution
int main(void) {
   int a[SIZE] = {0, 1, 2, 3, 4}; // initialize array a

   puts("Effects of passing entire array by reference:\n\nThe "
      "values of the original array are:");

   // output original array
   for (size_t i = 0; i < SIZE; ++i) {
      printf("%3d", a[i]);
   }

   puts(""); // outputs a newline

   modifyArray(a, SIZE); // pass array a to modifyArray by reference
   puts("The values of the modified array are:");

   // output modified array
   for (size_t i = 0; i < SIZE; ++i) {
      printf("%3d", a[i]);
   }

```

Fig. 6.11 | Passing arrays and individual array elements to functions. (Part 1 of 2.)

```c
   // output value of a[3]
   printf("\n\n\nEffects of passing array element "
      "by value:\n\nThe value of a[3] is %d\n", a[3]);

   modifyElement(a[3]); // pass array element a[3] by value

   // output value of a[3]
   printf("The value of a[3] is %d\n", a[3]);
}

// in function modifyArray, "b" points to the original array "a" in memory
void modifyArray(int b[], size_t size) {
   // multiply each array element by 2
   for (size_t j = 0; j < size; ++j) {
      b[j] *= 2; // actually modifies original array
   }
}

// in function modifyElement, "e" is a local copy of array element
// a[3] passed from main
void modifyElement(int e) {
   e *= 2; // multiply parameter by 2
   printf("Value in modifyElement is %d\n", e);
}
```

```
Effects of passing entire array by reference:

The values of the original array are:
  0  1  2  3  4
The values of the modified array are:
  0  2  4  6  8


Effects of passing array element by value:

The value of a[3] is 6
Value in modifyElement is 12
The value of a[3] is 6
```

Fig. 6.11 | Passing arrays and individual array elements to functions. (Part 2 of 2.)

Passing Arrays to Functions in C

- **3. Best Practice: Using const for Safety**
- **Purpose:** To prevent a function from accidentally modifying an array when it is only supposed to read from it (Principle of Least Privilege).
- **Syntax:**
  - Add the const qualifier before the array parameter in the function definition.
  - void printArray(const int b[], size_t size)
- **Result:**
  - The compiler will generate an **error** if the function attempts to change any value in the array b. This is a critical safety feature.

```
1  // in function tryToModifyArray, array b is const, so it cannot be
2  // used to modify its array argument in the caller
3  void tryToModifyArray(const int b[]) {
4     b[0] /= 2; // error
5     b[1] /= 2; // error
6     b[2] /= 2; // error
7  }
```

EXAMPLE: The Bubble Sort Method

- **Core Idea:** A simple sorting technique where smaller values gradually “bubble up” to the top of the array, while larger values “sink” to the bottom.
- **The Process:**
  - The algorithm makes multiple **passes** through the array.
  - On each pass, it compares **adjacent pairs** of elements (e.g., a[0] with a[1], then a[1] with a[2], etc.).
  - If a pair is in the wrong order, their values are **swapped**.
- **Key Outcome of Each Pass:**
  - After the first pass, the largest element is guaranteed to be in its final position at the end of the array.
  - After the second pass, the second-largest element is in its final position, and so on

```c
// fig06_12.c
// Sorting an array's values into ascending order.
#include <stdio.h>
#define SIZE 10

// function main begins program execution
int main(void) {
   int a[SIZE] = {2, 6, 4, 8, 10, 12, 89, 68, 45, 37};

   puts("Data items in original order");

   // output original array
   for (size_t i = 0; i < SIZE; ++i) {
      printf("%4d", a[i]);
   }

   // bubble sort
   // loop to control number of passes
   for (int pass = 1; pass < SIZE; ++pass) {
      // loop to control number of comparisons per pass
      for (size_t i = 0; i < SIZE - 1; ++i) {
         // compare adjacent elements and swap them if first
         // element is greater than second element
         if (a[i] > a[i + 1]) {
            int hold = a[i];
            a[i] = a[i + 1];
            a[i + 1] = hold;
         }
      }
   }

   puts("\nData items in ascending order");

   // output sorted array
   for (size_t i = 0; i < SIZE; ++i) {
      printf("%4d", a[i]);
   }

   puts("");
}
```

```
Data items in original order
   2   6   4   8  10  12  89  68  45  37
Data items in ascending order
   2   4   6   8  10  12  37  45  68  89
```

Fig. 6.12 | Sorting an array's values into ascending order. (Part 1 of 2.)

Fig. 6.12 | Sorting an array's values into ascending order. (Part 2 of 2.)

Case Study - Survey Data Analysis (The Setup)

- **Objective:** To analyze a set of survey data by calculating three key statistical measures: the Mean, Median, and Mode.
- **The Dataset:**
- An array containing 99 survey responses.
- Each response is an integer value from 1 to 9.
- **The Mean (Average)**
  - **Definition:** The arithmetic average of all the values.
- **The Median (Middle Value)**
  - **Definition:** The value that appears in the middle of a **sorted** dataset.
  - **Limitation:** The described function does not handle cases where there is an even number of elements.
- **The Mode (Most Frequent Value)**
  - **Definition:** The value that occurs most frequently in the dataset.

```
37 // calculate average of all response values
38 void mean(const int answer[]) {
39    printf("%s\n%s\n%s\n", "--------", "  Mean", "--------");
40
41    int total = 0; // variable to hold sum of array elements
42
43    // total response values
44    for (size_t j = 0; j < SIZE; ++j) {
45       total += answer[j];
46    }
47
48    printf("The mean is the average value of the data\n"
49           "items. The mean is equal to the total of\n"
50           "all the data items divided by the number\n"
51           "of data items (%u). The mean value for\n"
52           "this run is: %u / %u = %.4f\n\n",
53           SIZE, total, SIZE, (double) total / SIZE);
54 }
55
```

```
56  // sort array and determine median element's value
57  void median(int answer[]) {
58     printf("\n%s\n%s\n%s\n%s", "--------", " Median", "--------",
59            "The unsorted array of responses is");
60
61     printArray(answer); // output unsorted array
62
63     bubbleSort(answer); // sort array
64
65     printf("%s", "\n\nThe sorted array is");
66     printArray(answer); // output sorted array
67
68     // display median element
69     printf("\n\nThe median is element %u of\n"
70            "the sorted %u element array.\n"
71            "For this run the median is %u\n\n",
72            SIZE / 2, SIZE, answer[SIZE / 2]);
73  }
74
```

```
75  // determine most frequent response
76  void mode(int freq[], const int answer[]) {
77     printf("\n%s\n%s\n%s\n", "--------", "  Mode", "--------");
78
79     // initialize frequencies to 0
80     for (size_t rating = 1; rating <= 9; ++rating) {
81        freq[rating] = 0;
82     }
83
84     // summarize frequencies
85     for (size_t j = 0; j < SIZE; ++j) {
86        ++freq[answer[j]];
87     }
88
89     // output headers for result columns
90     printf("%s%11s%19s\n\n%54s\n%54s\n\n",
91            "Response", "Frequency", "Bar Chart",
92            "1    1    2    2", "5    0    5    0    5");
93
94     // output results
95     int largest = 0; // represents largest frequency
96     int modeValue = 0; // represents most frequent response
97
98     for (size_t rating = 1; rating <= 9; ++rating) {
99        printf("%8zu%11d          ", rating, freq[rating]);
100
101       // keep track of mode value and largest frequency value
102       if (freq[rating] > largest) {
103          largest = freq[rating];
104          modeValue = rating;
105       }
106
107       // output bar representing frequency value
108       for (int h = 1; h <= freq[rating]; ++h) {
109          printf("%s", "*");
110       }
111
112       puts(""); // being new line of output
113    }
114
115    // display the mode value
116    printf("\nThe mode is the most frequent value.\n"
117           "For this run the mode is %d which occurred %d times.\n",
118           modeValue, largest);
119 }
120
121 // function that sorts an array with bubble sort algorithm
122 void bubbleSort(int a[]) {
123    // loop to control number of passes
124    for (int pass = 1; pass < SIZE; ++pass) {
125       // loop to control number of comparisons per pass
126       for (size_t j = 0; j < SIZE - 1; ++j) {
127          // swap elements if out of order
128          if (a[j] > a[j + 1]) {
129             int hold = a[j];
130             a[j] = a[j + 1];
131             a[j + 1] = hold;
132          }
133       }
134    }
135 }
```

```c
// fig06_13.c
// Survey data analysis with arrays:
// computing the mean, median and mode of t
#include <stdio.h>
#define SIZE 99

// function prototypes
void mean(const int answer[]);
void median(int answer[]);
void mode(int freq[], const int answer[]) ;
void bubbleSort(int a[]);
void printArray(const int a[]);

// function main begins program execution
int main(void) {
   int frequency[10] = {0}; // initialize array frequency

   // initialize array response
   int response[SIZE] =
      {6, 7, 8, 9, 8, 7, 8, 9, 8, 9,
       7, 8, 9, 5, 9, 8, 7, 8, 7, 8,
       6, 7, 8, 9, 3, 9, 8, 7, 8, 7,
       7, 8, 9, 8, 9, 8, 9, 7, 8, 9,
       6, 7, 8, 7, 8, 7, 9, 8, 9, 2,
       7, 8, 9, 8, 9, 8, 9, 7, 5, 3,
       5, 6, 7, 2, 5, 3, 9, 4, 6, 4,
       7, 8, 9, 6, 8, 7, 8, 9, 7, 8,
       7, 4, 4, 2, 5, 3, 8, 7, 5, 6,
       4, 5, 6, 1, 6, 5, 7, 8, 7};

   // process responses
   mean(response);
   median(response);
   mode(frequency, response);
}
```

```
--------
  Mean
--------
The mean is the average value of the data
items. The mean is equal to the total of
all the data items divided by the number
of data items (99). The mean value for
this run is: 681 / 99 = 6.8788


--------
 Median
--------
The unsorted array of responses is
 6 7 8 9 8 7 8 9 8 9 7 8 9 5 9 8 7 8 7 8
 6 7 8 9 3 9 8 7 8 7 7 8 9 8 9 8 9 7 8 9
```

Fig. 6.13 | Survey data analysis with arrays: computing the mean, median and mode of the data. (Part 4 of 5.)

```
 6 7 8 7 8 7 9 8 9 2 7 8 9 8 9 8 9 7 5 3
 5 6 7 2 5 3 9 4 6 4 7 8 9 6 8 7 8 9 7 8
 7 4 4 2 5 3 8 7 5 6 4 5 6 1 6 5 7 8 7

The sorted array is
 1 2 2 2 3 3 3 3 4 4 4 4 4 5 5 5 5 5 5 5
 5 6 6 6 6 6 6 6 6 6 7 7 7 7 7 7 7 7 7 7
 7 7 7 7 7 7 7 7 7 7 7 7 7 8 8 8 8 8 8 8
 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8
 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9

The median is element 49 of
the sorted 99 element array.
For this run the median is 7


--------
  Mode
--------
Response  Frequency          Bar Chart

                                    1    1    2    2
                               5    0    5    0    5

       1          1          *
       2          3          ***
       3          4          ****
       4          5          *****
       5          8          ********
       6          9          *********
       7         23          ***********************
       8         27          ***************************
       9         19          *******************

The mode is the most frequent value.
For this run the mode is 8 which occurred 27 times.
```

Searching Arrays - The Linear Search

- **What is Searching?** The process of finding a specific **key value** within an array. The goal is to determine if the value exists and, if so, at what location (index).
- **Technique 1: Linear Search**
- **How it Works:**
  - The simplest search method.
  - It compares the search key to **every single element** in the array, one by one, starting from the beginning.
- **The Process:**
  - Start at index 0.
  - Compare the element to the search key.
  - If it matches, return the index.
  - If it doesn’t match, move to the next element and repeat.
  - If the end of the array is reached without a match, return -1 (an invalid index) to indicate the key was not found.

```c
// fig06_14.c
// Linear search of an array.
#include <stdio.h>
#define SIZE 100

// function prototype
int linearSearch(const int array[], int key, size_t size);

// function main begins program execution
int main(void) {
   int a[SIZE] = {0}; // create array a

   // create some data
   for (size_t x = 0; x < SIZE; ++x) {
      a[x] = 2 * x;
   }

   printf("Enter integer search key: ");
   int searchKey = 0; // value to locate in array a
   scanf("%d", &searchKey);

   // attempt to locate searchKey in array a
   int subscript = linearSearch(a, searchKey, SIZE);

   // display results
   if (subscript != -1) {
      printf("Found value at subscript %d\n", subscript);
   }
   else {
      puts("Value not found");
   }
}
```

```c
// compare key to every element of array until the location is found
// or until the end of array is reached; return subscript of element
// if key is found or -1 if key is not found
int linearSearch(const int array[], int key, size_t size) {
   // loop through array
   for (size_t n = 0; n < size; ++n) {
      if (array[n] == key) {
         return n; // return location of key
      }
   }

   return -1; // key not found
}
```

```
Enter integer search key: 36
Found value at subscript 18
```

Fig. 6.14 | Linear search of an array. (Part 1 of 2.)

```
Enter integer search key: 37
Value not found
```

Fig. 6.14 | Linear search of an array. (Part 2 of 2.)

Searching Arrays - The Binary Search

- **Technique 2: Binary Search (The High-Speed Method)**
- **CRITICAL REQUIREMENT:** The array **MUST** be sorted before performing a binary search.
- **How it Works (The "Divide and Conquer" Approach):**
  - The algorithm repeatedly divides the search interval in half.
  - Compare the search key to the **middle** element of the current array segment.
  - If they are equal, the key is found.
  - If the search key is **less** than the middle element, search the **first half**.
  - If the search key is **greater** than the middle element, search the **second half**.
  - Repeat this process on the smaller sub-array until the key is found or the sub-array is empty.

- **Performance: The Power of Binary Search**
- This method eliminates half the remaining elements with each comparison, making it incredibly fast.

Array Size	Linear Search (Avg. Comparisons)	Binary Search (Max Comparisons)
~1,000 elements	~500	**10**
~1,000,000 elements	~500,000	**20**
1 Billion elements	~500,000,000	**30**

```c
// fig06_15.c
// Binary search of a sorted array.
#include <stdio.h>
#define SIZE 15

// function prototypes
int binarySearch(const int b[], int key, size_t low, size_t high);
void printHeader(void);
void printRow(const int b[], size_t low, size_t mid, size_t high);

// function main begins program execution
int main(void) {
   int a[SIZE] = {0}; // create array a

   // create data
   for (size_t i = 0; i < SIZE; ++i) {
      a[i] = 2 * i;
   }

   printf("%s", "Enter a number between 0 and 28: ");
   int key = 0; // value to locate in array a
   scanf("%d", &key);

   printHeader();

   // search for key in array a
   int result = binarySearch(a, key, 0, SIZE - 1);

   // display results
   if (result != -1) {
      printf("\n%d found at subscript %d\n", key, result);
   }
   else {
      printf("\n%d not found\n", key);
   }
}
```

```c
// Print a header for the output
void printHeader(void) {
   puts("\nSubscripts:");

   // output column head
   for (int i = 0; i < SIZE; ++i) {
      printf("%3d ", i);
   }

   puts(""); // start new line of output

   // output line of - characters
   for (int i = 1; i <= 4 * SIZE; ++i) {
      printf("%s", "-");
   }

   puts(""); // start new line of output
}
```

```
37
38  // function to perform binary search of an array
39  int binarySearch(const int b[], int key, size_t low, size_t high) {
40     // loop until low subscript is greater than high subscript
41     while (low <= high) {
42        size_t middle = (low + high) / 2; // determine middle subscript
43
44        // display subarray used in this loop iteration
45        printRow(b, low, middle, high);
46
47        // if key matches, return middle subscript
48        if (key == b[middle]) {
49           return middle;
50        }
51        else if (key < b[middle]) { // if key < b[middle], adjust high
52           high = middle - 1; // next iteration searches low end of array
53        }
54        else { // key > b[middle], so adjust low
55           low = middle + 1; // next iteration searches high end of array
56        }
57     } // end while
58
59     return -1; // searchKey not found
60  }
```

```
81  // Print one row of output showing the current
82  // part of the array being processed.
83  void printRow(const int b[], size_t low, size_t mid, size_t high) {
84     // loop through entire array
85     for (size_t i = 0; i < SIZE; ++i) {
86        // display spaces if outside current subarray range
87        if (i < low || i > high) {
88           printf("%s", "    ");
89        }

90        else if (i == mid) { // display middle element
91           printf("%3d*", b[i]); // mark middle value
92        }
93        else { // display other elements in subarray
94           printf("%3d ", b[i]);
95        }
96     }
97
98     puts(""); // start new line of output
99  }
```

```
Enter a number between 0 and 28: 25

Subscripts:
  0   1   2   3   4   5   6   7   8   9  10  11  12  13  14
---------------------------------------------------------------
  0   2   4   6   8  10  12  14* 16  18  20  22  24  26  28
                              16  18  20  22* 24  26  28
                                          24  26* 28
                                          24*

25 not found
```

```
Enter a number between 0 and 28: 8

Subscripts:
  0   1   2   3   4   5   6   7   8   9  10  11  12  13  14
---------------------------------------------------------------
  0   2   4   6   8  10  12  14* 16  18  20  22  24  26  28
  0   2   4   6*  8  10  12
                  8  10* 12
                  8*

8 found at subscript 4
```

```
Enter a number between 0 and 28: 6

Subscripts:
  0   1   2   3   4   5   6   7   8   9  10  11  12  13  14
---------------------------------------------------------------
  0   2   4   6   8  10  12  14* 16  18  20  22  24  26  28
  0   2   4   6*  8  10  12

6 found at subscript 3
```

Fig. 6.15 | Binary search of a sorted array. (Part 3 of 3.)

Multidimensional Arrays - The Basics

- **What is a Two-Dimensional Array?**
- An array with multiple subscripts, used to represent tables of values with **rows** and **columns**.
- An array with m rows and n columns is called an **m-by-n array**.
- **Declaration and Access**
- **Declaration:** dataType arrayName[numberOfRows][numberOfColumns];
- **Accessing an Element:** arrayName[rowIndex][columnIndex];
  - By convention, the **first** subscript identifies the **row**.
  - The **second** subscript identifies the **column**

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Initialization & Passing to Functions

- **Initializing a 2D Array**
- You can initialize an array when you define it using nested braces {} for each row.
- int b[2][2] = {{1, 2}, {3, 4}}; *// Row 0 is {1, 2}, Row 1 is {3, 4}*
- **Partial Initialization:**
  - Any elements not explicitly assigned a value are automatically initialized to **0**.
- *// Initializes b[0][0]=1, b[0][1]=0, b[1][0]=3, b[1][1]=4*
- int b[2][2] = {{1}, {3, 4}};
- **Passing 2D Arrays to Functions**
- **The Rule:** When declaring a function parameter for a 2D array, the size of the **first dimension (rows) can be omitted**, but **all subsequent dimensions (columns) MUST be specified**.
- **Syntax:**
- *// Correct function definition for an array with 3 columns*
- void printArray(int a[][3], size_t rows);
- **Why?** The compiler needs the column size to calculate the memory address of elements. To find a[1][0], it needs to know how many elements to “skip” in the first row.

The Key Tool: Nested for Loops

```
int total = 0;
// Outer loop iterates through the rows
for (int row = 0; row < 3; ++row) {
    // Inner loop iterates through columns of the current row
    for (int column = 0; column < 4; ++column) {
        total += a[row][column];
    }
}
```

```c
// fig06_17.c
// Two-dimensional array manipulations.
#include <stdio.h>
#define STUDENTS 3
#define EXAMS 4

// function prototypes
int minimum(const int grades[][EXAMS], size_t pupils, size_t tests);
int maximum(const int grades[][EXAMS], size_t pupils, size_t tests);
double average(const int setOfGrades[], size_t tests);
void printArray(const int grades[][EXAMS], size_t pupils, size_t tests);

// function main begins program execution
int main(void) {
   // initialize student grades for three students (rows)
   int studentGrades[STUDENTS][EXAMS] =
      {{77, 68, 86, 73},
       {96, 87, 89, 78},
       {70, 90, 86, 81}};

   // output array studentGrades
   puts("The array is:");
   printArray(studentGrades, STUDENTS, EXAMS);

   // determine smallest and largest grade values
   printf("\n\nLowest grade: %d\nHighest grade: %d\n",
      minimum(studentGrades, STUDENTS, EXAMS),
      maximum(studentGrades, STUDENTS, EXAMS));

   // calculate average grade for each student
   for (size_t student = 0; student < STUDENTS; ++student) {
      printf("The average grade for student %zu is %.2f\n",
         student, average(studentGrades[student], EXAMS));
   }
}
```

```
36
37 // Find the minimum grade
38 int minimum(const int grades[][EXAMS], size_t pupils, size_t tests) {
39    int lowGrade = 100; // initialize to highest possible grade
40
41    // loop through rows of grades
42    for (size_t row = 0; row < pupils; ++row) {
43       // loop through columns of grades
44       for (size_t column = 0; column < tests; ++column) {
45          if (grades[row][column] < lowGrade) {
46             lowGrade = grades[row][column];
47          }
48       }
49    }
50
51    return lowGrade; // return minimum grade
52 }
```

```
54 // Find the maximum grade
55 int maximum(const int grades[][EXAMS], size_t pupils, size_t tests) {
56    int highGrade = 0; // initialize to lowest possible grade
57
58    // loop through rows of grades
59    for (size_t row = 0; row < pupils; ++row) {
60       // loop through columns of grades
61       for (size_t column = 0; column < tests; ++column) {
62          if (grades[row][column] > highGrade) {
63             highGrade = grades[row][column];
64          }
65       }
66    }
67
68    return highGrade; // return maximum grade
69 }
```

```
71 // Determine the average grade for a particular student
72 double average(const int setOfGrades[], size_t tests) {
73    int total = 0; // sum of test grades
74
75    // total all grades for one student
76    for (size_t test = 0; test < tests; ++test) {
77       total += setOfGrades[test];
78    }
79
80    return (double) total / tests; // average
81 }
82
```

```
83  // Print the array
84  void printArray(const int grades[][EXAMS], size_t pupils, size_t tests) {
85     // output column heads
86     printf("%s", "                  [0]  [1]  [2]  [3]");
87
88     // output grades in tabular format
89     for (size_t row = 0; row < pupils; ++row) {
90        // output label for row
91        printf("\nstudentGrades[%zu] ", row);
92
93        // output grades for one student
94        for (size_t column = 0; column < tests; ++column) {
95           printf("%-5d", grades[row][column]);
96        }
97     }
98  }
```

```
The array is:
                  [0]  [1]  [2]  [3]
studentGrades[0] 77   68   86   73
studentGrades[1] 96   87   89   78
studentGrades[2] 70   90   86   81

Lowest grade: 68
Highest grade: 96
The average grade for student 0 is 76.00
The average grade for student 1 is 87.50
The average grade for student 2 is 81.75
```

Variable-Length Arrays (VLAs) - The Concept

- **The Problem:** How do you create an array when you don’t know its required size until the program is already running?
- **The Solution: Variable-Length Arrays (VLAs)** A VLA is an array whose size is determined by an expression evaluated at **execution time**, not at compile time.
  Creating a VLA
- The syntax is the same as a regular array, but you use a variable for the size.

```
int arraySize;
printf("Enter the number of elements: ");
scanf("%d", &arraySize);

// The VLA is created here using a variable size
int data[arraySize];
```

```c
// fig06_18.c
// Using variable-length arrays in C99
#include <stdio.h>

// function prototypes
void print1DArray(size_t size, int array[size]);
void print2DArray(size_t row, size_t col, int array[row][col]);

int main(void) {
   printf("%s", "Enter size of a one-dimensional array: ");
   int arraySize = 0; // size of 1-D array
   scanf("%d", &arraySize);

   int array[arraySize]; // declare 1-D variable-length array

   printf("%s", "Enter number of rows and columns in a 2-D array: ");
   int row1 = 0; // number of rows in a 2-D array
   int col1 = 0; // number of columns in a 2-D array
   scanf("%d %d", &row1, &col1);

   int array2D1[row1][col1]; // declare 2-D variable-length array

   printf("%s",
      "Enter number of rows and columns in another 2-D array: ");
   int row2 = 0; // number of rows in a 2-D array
   int col2 = 0; // number of columns in a 2-D array
   scanf("%d %d", &row2, &col2);

   int array2D2[row2][col2]; // declare 2-D variable-length array

   // test sizeof operator on VLA
   printf("\nsizeof(array) yields array size of %zu bytes\n",
      sizeof(array));

   // assign elements of 1-D VLA
   for (size_t i = 0; i < arraySize; ++i) {
      array[i] = i * i;
   }

   // assign elements of first 2-D VLA
   for (size_t i = 0; i < row1; ++i) {
      for (size_t j = 0; j < col1; ++j) {
         array2D1[i][j] = i + j;
      }
   }

   // assign elements of second 2-D VLA
   for (size_t i = 0; i < row2; ++i) {
      for (size_t j = 0; j < col2; ++j) {
         array2D2[i][j] = i + j;
      }
   }

   puts("\nOne-dimensional array:");
   print1DArray(arraySize, array); // pass 1-D VLA to function

   puts("\nFirst two-dimensional array:");
   print2DArray(row1, col1, array2D1); // pass 2-D VLA to function

   puts("\nSecond two-dimensional array:");
   print2DArray(row2, col2, array2D2); // pass other 2-D VLA to function
}

void print1DArray(size_t size, int array[size]) {
   // output contents of array
   for (size_t i = 0; i < size; i++) {
      printf("array[%zu] = %d\n", i, array[i]);
   }
}
```

```c
void print2DArray(size_t row, size_t col, int array[row][col]) {
   // output contents of array
   for (size_t i = 0; i < row; ++i) {
      for (size_t j = 0; j < col; ++j) {
         printf("%5d", array[i][j]);
      }

      puts("");
   }
}
```

```
Enter size of a one-dimensional array: 6
Enter number of rows and columns in a 2-D array: 2 5
Enter number of rows and columns in another 2-D array: 4 3

sizeof(array) yields array size of 24 bytes

One-dimensional array:
array[0] = 0
array[1] = 1
array[2] = 4
array[3] = 9
array[4] = 16
array[5] = 25

First two-dimensional array:
    0    1    2    3    4
    1    2    3    4    5
```

Fig. 6.18 | Using variable-length arrays in C99. (Part 2 of 3.)